

Pró-Reitoria Acadêmica
Escola de Educação, Tecnologia e Comunicação
Curso de Bacharelado em Engenharia de Software
Trabalho da Disciplina de Teste de Software

CashLens - MVP de Gestão Financeira

Relatório Final de Testes

Versão 1.0

**Autores: João Marcos Azevedo Cruz e João Pedro
Tavares Teixeira**

HISTÓRICO DE REVISÃO

Data	Versão	Responsável	Descrição
27/05/2026	1.0	Integrantes CashLens	Consolidação do relatório final de testes do projeto CashLens.

LINKS DOS REPOSITÓRIOS

Repositório backend: github.com/CashLens-Project/cashlens-api

Repositório frontend: github.com/CashLens-Project/CashLens-Front

Repositório documentação: github.com/CashLens-Project/CashLens_Docs_Teste_Software

1. INTRODUÇÃO AO SISTEMA DESENVOLVIDO

O CashLens é um sistema de gestão financeira desenvolvido como projeto final, com foco em apoiar o controle de receitas, despesas, vendas, importações financeiras e indicadores gerenciais. A proposta do sistema é oferecer uma visão mais clara da situação financeira do negócio, permitindo que o usuário acompanhe resultados, registre operações, importe dados externos e consulte demonstrativos de apoio a tomada de decisão.

No uso principal, o usuário acessa o sistema de forma autenticada, importa ou registra informações financeiras e acompanha os dados consolidados em telas como dashboard, fluxo de caixa, DRE, waterfall e operações. O sistema também possui uma assistente de IA integrada ao contexto financeiro, utilizada como apoio para consultas em linguagem natural, sem substituir as regras e dados oficiais mantidos pelo backend. A solução foi organizada em três frentes principais:

- **Frontend:** React, responsável pela interface do usuário, navegação entre telas, exibição dos indicadores, formulários de operações, importação de arquivos, filtros de período e interação com a assistente;
- **Backend:** Spring Boot, responsável pelas APIs REST, regras de negócio, autenticação, autorização, processamento de importações, validações, cálculos financeiros, integridade dos dados e integração com o banco;
- **Banco:** PostgreSQL com suporte a pgvector, utilizado para persistência dos dados financeiros, histórico de importações, vínculo com usuários autenticados e estruturas de apoio da camada de IA.
- **Autenticação:** utiliza Keycloak com tokens JWT, dessa forma, os endpoints protegidos devem responder apenas para usuários autenticados, mantendo o isolamento dos dados financeiros de cada usuário.

2. PLANEJAMENTO DE TESTES

O planejamento dos testes foi definido para cobrir as funcionalidades mais relevantes do CashLens, combinando testes automatizados, testes de API e verificações manuais no navegador. A estratégia adotada buscou validar tanto regras internas do sistema quanto fluxos percebidos pelo usuário final, foram planejados os seguintes grupos de teste:

Grupo de teste	Objetivo
Testes unitários	Validar regras de negócio, validadores, parsers, segurança local, cálculos financeiros e comportamentos isolados.
Testes com Testcontainers	Validar fluxos que dependem de banco realista, migrações Flyway, persistência, idempotência, rollback e consultas financeiras.
Testes de API com Postman/Swagger	Validar endpoints do backend com token JWT, sem token, payloads válidos, payloads inválidos e respostas HTTP.

Testes E2E manuais em navegador	Simular a navegação do usuário pelo sistema, avaliando login, telas financeiras, importação, responsividade e uso da IA.
Testes de regressão	Confirmar que comportamentos corrigidos continuam funcionando após ajustes no sistema.

2.1 Ambiente Planejado

Item	Descricao
Backend	Spring Boot com Java 17
Frontend	React
Banco dos testes	PostgreSQL/pgvector em Testcontainers
Autenticação	Keycloak
Automação de testes	JUnit 5, Mockito, Spring Boot Test
Testes de API	Postman e Swagger
Navegador	Navegador atualizado para validações manuais

2.2 Critérios Gerais

Um teste foi considerado aprovado quando o comportamento observado correspondeu ao comportamento esperado, sem falhas técnicas, sem exposição indevida de dados e com resposta coerente para o usuário. Quando algum comportamento inadequado foi encontrado durante a execução, ele foi registrado, corrigido e posteriormente coberto pelos testes de regressão.

3. DESCRIÇÃO DOS CASOS DE TESTE

Para o relatório final foram selecionados cinco casos de teste principais. Eles representam os fluxos mais importantes do sistema: acesso seguro, entrada de dados, persistência, consulta financeira e uso da IA.

CT01 - Login e Acesso Autenticado

Objetivo: validar se o usuário consegue acessar o sistema com credenciais validas e se os endpoints protegidos bloqueiam requisições sem token.

Passos principais:

1. Acessar a tela de login.
2. Informar credenciais validas.
3. Confirmar o acesso ao sistema.
4. Obter um token JWT valido.
5. Chamar endpoint protegido com token.
6. Repetir a chamada sem token.

Dados de entrada:

Campo	Exemplo
Usuário	Usuário de teste cadastrado no Keycloak
Senha	Senha valida do usuário
Endpoint	GET /finance/dre?mes=2026-06
Autorização	Authorization: Bearer <accessToken>

Saida esperada:

- com credenciais validas, o usuário acessa a área autenticada;
- com token valido, a API retorna dados financeiros do usuário;
- sem token, a API retorna 401 Unauthorized ou 403 Forbidden.

Critério de sucesso: o sistema deve permitir acesso apenas a usuários autenticados e não deve expor dados financeiros em requisições anônimas.

CT02 - Preview de Importação Financeira

Objetivo: validar se o sistema permite enviar arquivo financeiro aceito e gerar uma pré-visualização antes da aplicação definitiva dos dados.

Passos principais:

1. Abrir o Postman/Swagger.
2. Configurar POST /ingest/previews.
3. Informar token JWT valido.
4. Enviar arquivo financeiro em multipart/form-data.
5. Informar tipo e modo.
6. Verificar a resposta da API.

Dados de entrada:

Campo	Exemplo
Arquivo	.csv, .xls ou .xlsx aceito pelo sistema
tipo	vendas ou despesas
modo	INCREMENTAL

Saida esperada:

- API retorna status de sucesso;
- resposta contém a pré-visualização da importação;
- dados ainda não são aplicados definitivamente;
- registros inválidos são sinalizados ao usuário.

Critério de sucesso: o usuário deve conseguir revisar os dados antes da gravação definitiva.

CT03 - Aplicação da Importação e Integridade dos Dados

Objetivo: validar se, após a confirmação da importação, os dados financeiros são persistidos corretamente e respeitam as regras de integridade.

Passos principais:

1. Executar de testes de integração.
2. Subir banco PostgreSQL/pgvector via Testcontainers.
3. Aplicar migrações Flyway.
4. Criar preview de importação.
5. Confirmar a aplicação dos dados.
6. Consultar os registros persistidos.

Dados de entrada:

Campo	Exemplo
Banco	PostgreSQL/pgvector temporario
Mes	2026-06
Tipo	Vendas ou despesas
Modo	INCREMENTAL

Saida esperada:

- migrações aplicadas sem erro;
- registros gravados no banco de teste;
- valores persistidos coerentes com a massa de entrada;
- importações repetidas não geram duplicidade indevida;
- regras de integridade permanecem validas.

Critério de sucesso: os dados devem ser gravados corretamente no banco temporário e continuar disponíveis para consulta posterior.

CT04 - Consulta da DRE do Mes

Objetivo: validar se o sistema retorna corretamente a DRE do mês selecionado e se os valores exibidos no frontend são coerentes com a API.

Passos principais:

1. Obter token JWT valido.
2. Executar GET /finance/dre?mes=2026-06 no Postman/Swagger.
3. Verificar status HTTP e campos financeiros retornados.
4. Acessar a tela de DRE no frontend.
5. Selecionar o mesmo mês.
6. Comparar os dados da tela com a resposta da API.

Dados de entrada:

Campo	Exemplo
Endpoint	GET /finance/dre
Parâmetro	mes=2026-06
Autorização	Bearer <accessToken>

Saida esperada:

- status 200 OK;
- retorno com receita liquida, COGS, despesas e resultado operacional;
- tela de DRE exibe valores coerentes com o backend.

Critério de sucesso: a API e a interface devem apresentar informações financeiras consistentes para o mesmo período.

CT05 - IA Consultando Dado Financeiro Real

Objetivo: validar se a assistente de IA responde perguntas financeiras usando dados controlados pelo backend, sem inventar valores e sem acessar dados de outro usuário.

Passos principais:

1. Acessar o sistema com usuário autenticado.
2. Abrir o widget da IA.
3. Enviar uma pergunta financeira objetiva.
4. Aguardar a resposta.
5. Comparar o valor informado com o endpoint financeiro correspondente, quando aplicável.
6. Verificar se não houve exposição de token, SQL, prompt interno ou dados de outro usuário.

Dados de entrada:

Campo	Exemplo
-------	---------

Pergunta	Qual minha receita líquida no mês de junho?
Mes	2026-06
Usuário	Usuário autenticado com dados no período

Saida esperada:

- resposta em linguagem clara;
- valores coerentes com o backend;
- uso de ferramentas financeiras controladas quando a pergunta depender de dado real;
- ausência de vazamento de informações internas ou sensíveis.

Critério de sucesso: a IA deve atuar como interface de linguagem natural, mas os dados financeiros oficiais devem continuar sendo fornecidos pelo backend.

4. RESULTADOS DAS EXECUÇÕES E EVIDÊNCIAS

As execuções foram realizadas no backend Spring e em validações manuais de API/interface.

4.1 Resumo das Execuções Automatizadas

Grupo	Comando	Resultado	Casos executados	Falhas
Testes unitários, serviço, segurança e IA	<code>./mvnw test</code>	Aprovado	129	0
Testes de integração com banco via Testcontainers	<code>./mvnw -Dtest="*IT" test</code>	Aprovado	16	0

Observação: Na versão atual da API não foi encontrado defeitos por meio dos testes, porém foi registrado vários durante o desenvolvimento, como mostrado nas evidências.

4.2 Testes de API, Swagger e Frontend

Grupo	Ferramenta	Objetivo	Resultado esperado
API autenticada	Postman/Swagger	Validar chamada com JWT válido.	Status de sucesso e retorno dos dados do usuário autenticado.
API sem token	Postman/Swagger	Validar bloqueio de acesso anônimo.	Status 401 ou 403, sem dados financeiros.
Fluxo visual	Navegador	Validar login, dashboards, operações, DRE, waterfall e IA.	Telas carregadas, fluxo navegável e informações coerentes.

4.3 Evidências

Evidência 1 - Testes automatizados unitários e de serviço

Objetivo: demonstrar a execução dos testes automatizados do backend.

```
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.057 s -- in io.project.cashlens.service.ingest
.CsvIngestionOrchestratorTest
[INFO] Running io.project.cashlens.service.ingest.FinancialDataPersistenceServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.079 s -- in io.project.cashlens.service.ingest
.FinancialDataPersistenceServiceTest
[INFO] Running io.project.cashlens.service.ingest.IngestionConflictServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in io.project.cashlens.service.ingest
.IngestionConflictServiceTest
[INFO] Running io.project.cashlens.service.ingest.IngestJobHistoryServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.072 s -- in io.project.cashlens.service.ingest
.IngestJobHistoryServiceTest
[INFO] Running io.project.cashlens.service.ingest.PreviewApplyServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.045 s -- in io.project.cashlens.service.ingest
.PreviewApplyServiceTest
[INFO] Running io.project.cashlens.service.ingest.RollbackJobServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s -- in io.project.cashlens.service.ingest
.RollbackJobServiceTest
[INFO] Running io.project.cashlens.service.manual.ManualAdjustmentIntegrityTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.060 s -- in io.project.cashlens.service.manual
.ManualAdjustmentIntegrityTest
[INFO] Running io.project.cashlens.service.manual.ManualAdjustmentServiceTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 s -- in io.project.cashlens.service.manua
l.ManualAdjustmentServiceTest
[INFO] Running io.project.cashlens.service.parser.CsvIngestionParserTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.089 s -- in io.project.cashlens.service.parser
.CsvIngestionParserTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 129, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.138 s
[INFO] Finished at: 2026-05-28T13:50:55-03:00
[INFO] -----
```

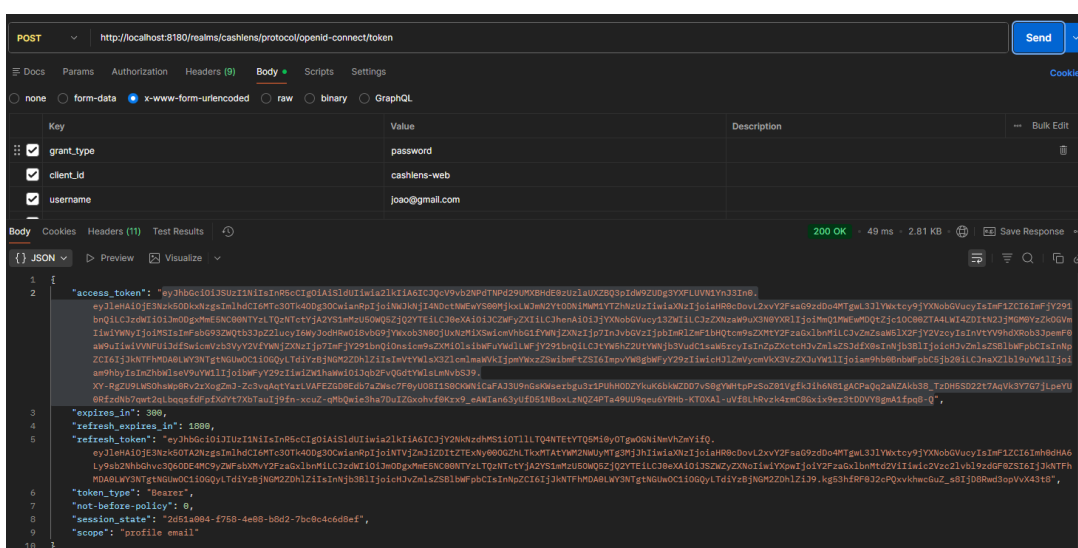
Evidência 2 - Testes de integração com banco em Testcontainers

Objetivo: demonstrar a execução dos testes que usam banco PostgreSQL/pgvector temporário e execução de migrations.

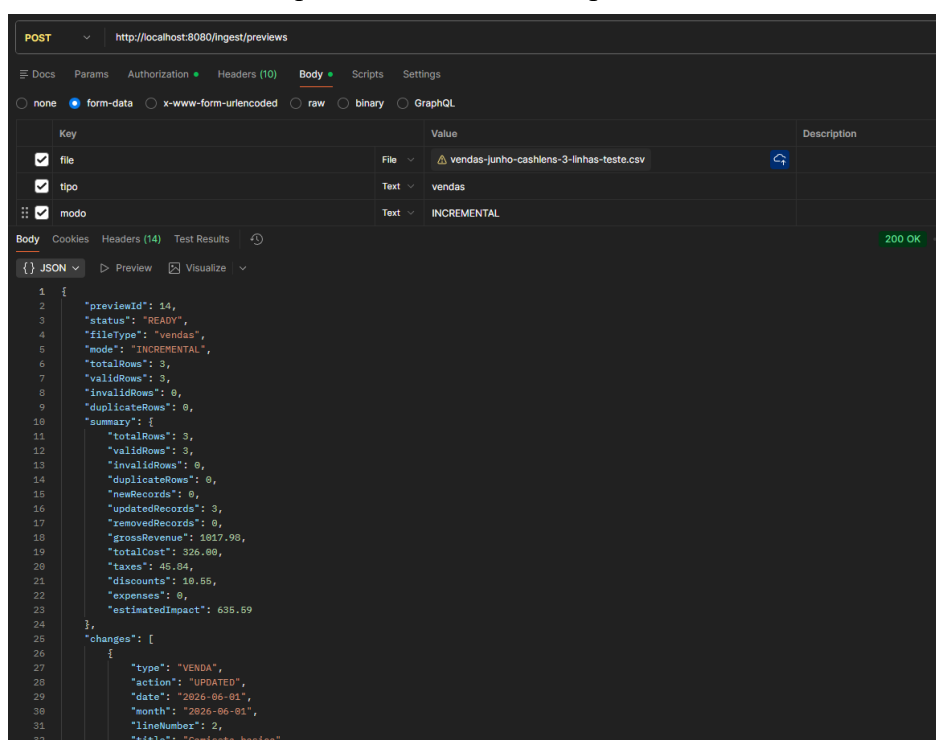
```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.358 s -- in io.project.cashlens.integration.in
gest.IngestPreviewApplyIT
[INFO] Running io.project.cashlens.integration.ingest.IngestPreviewControllerIT
2026-05-28T13:53:04.025-03:00 INFO 1160 --- [cashlens] [main] t.c.s.AnnotationConfigContextLoaderUtils : Co
uld not detect default configuration classes for test class [io.project.cashlens.integration.ingest.IngestPreviewContro
llerIT]: IngestPreviewControllerIT does not declare any static, non-private, non-final, nested classes annotated with @
Configuration.
2026-05-28T13:53:04.031-03:00 INFO 1160 --- [cashlens] [main] .b.t.c.SpringBootTestContextBootstrapper : Fo
und @SpringBootTestConfiguration io.project.cashlens.CashlensApplication for test class io.project.cashlens.integration.ing
est.IngestPreviewControllerIT
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.130 s -- in io.project.cashlens.integration.in
gest.IngestPreviewControllerIT
[INFO] Running io.project.cashlens.integration.ingest.IngestRollbackReliableIT
2026-05-28T13:53:04.157-03:00 INFO 1160 --- [cashlens] [main] t.c.s.AnnotationConfigContextLoaderUtils : Co
uld not detect default configuration classes for test class [io.project.cashlens.integration.ingest.IngestRollbackRelia
bleIT]: IngestRollbackReliableIT does not declare any static, non-private, non-final, nested classes annotated with @Co
nfiguration.
2026-05-28T13:53:04.161-03:00 INFO 1160 --- [cashlens] [main] .b.t.c.SpringBootTestContextBootstrapper : Fo
und @SpringBootTestConfiguration io.project.cashlens.CashlensApplication for test class io.project.cashlens.integration.ing
est.IngestRollbackReliableIT
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.311 s -- in io.project.cashlens.integration.in
gest.IngestRollbackReliableIT
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:09 min
[INFO] Finished at: 2026-05-28T13:53:04-03:00
[INFO] -----
```

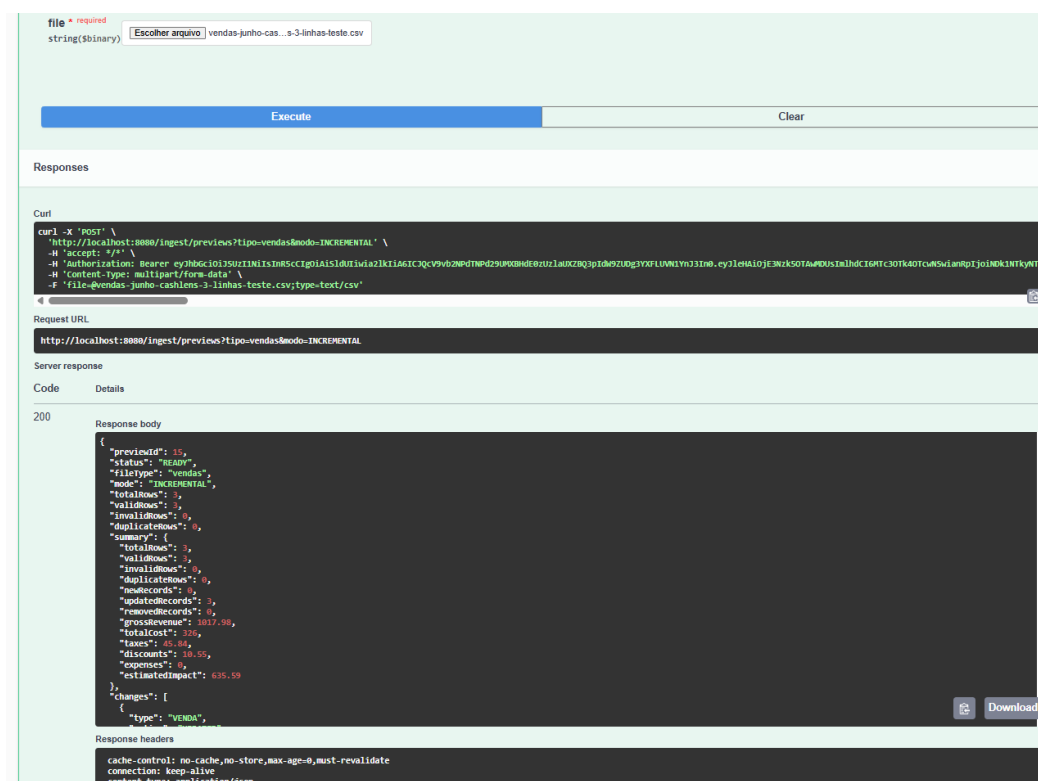
Evidência 3 - API bloqueando requisição sem token no Swagger/Postman

Objetivo: demonstrar que o keycloak emite adequadamente o token JWT ao validar o usuário.



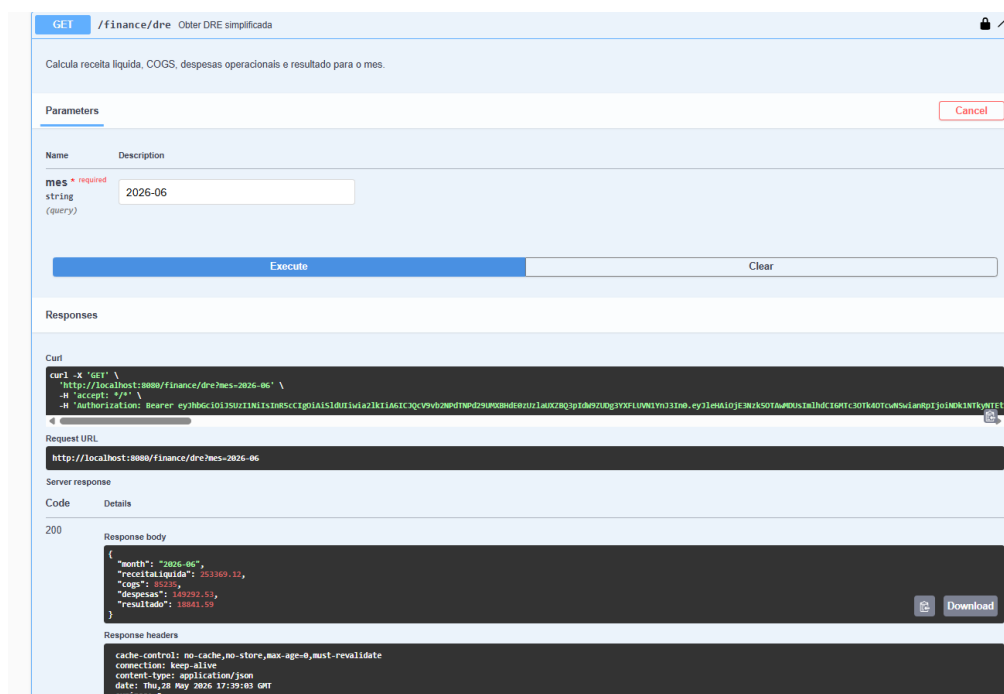
Objetivo: demonstrar o envio de arquivo financeiro aceito pelo sistema e o retorno da API.

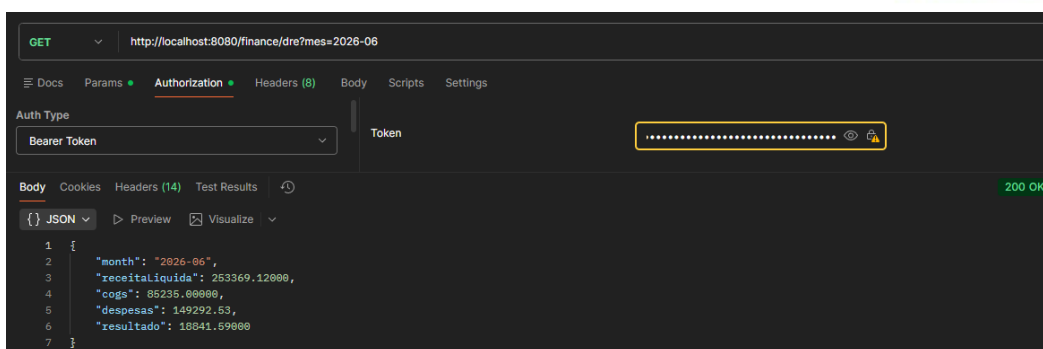




Evidência 7 – Verificação do DRE via Swagger/Postman

Objetivo: demonstrar a funcionalidade de DRE na API.





Evidência 8 – Exemplos de Defeitos encontrados durante o Desenvolvimento

Objetivo: demonstrar defeitos encontrados durante verificações de testes:

```

at org.apache.maven.surefire.junitplatform.JUnitPlatformProvider.invokeAllTests(JUnitPlatformProvider.java:168)
at org.apache.maven.surefire.junitplatform.JUnitPlatformProvider.invoke(JUnitPlatformProvider.java:136)
at org.apache.maven.surefire.booter.ForkedBooter.runSuitesInProcess(ForkedBooter.java:385)
at org.apache.maven.surefire.booter.ForkedBooter.execute(ForkedBooter.java:162)
at org.apache.maven.surefire.booter.ForkedBooter.run(ForkedBooter.java:507)
at org.apache.maven.surefire.booter.ForkedBooter.main(ForkedBooter.java:495)

[INFO]
[ERROR] Tests run: 10, Failures: 10, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 18.612 s
[INFO] Finished at: 2026-05-19T17:40:07-03:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.5.5:test (default-test) on project cashlens: There are test failures
.
[ERROR] See /mnt/c/Users/joao.acruz/Downloads/projectTest/spring/target/surefire-reports for the individual test results.
[ERROR] See dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
  
```

5. LIÇÕES APRENDIDAS

A execução dos testes demonstrou que o CashLens possui cobertura em diferentes camadas: regras internas, serviços, integração com banco, API e fluxo visual do usuário. A combinação dessas abordagens permitiu validar tanto comportamentos isolados quanto cenários mais próximos do uso real. As principais lições aprendidas foram:

- testes unitários e de serviço ajudam a validar regras de negócio com rapidez e baixo custo;
- testes com Testcontainers aumentam a confiabilidade por usarem banco temporário semelhante ao ambiente real;
- Postman e Swagger facilitam a validação dos contratos da API e dos fluxos autenticados;
- validações manuais no navegador continuam importantes para confirmar navegação, layout, responsividade e experiência do usuário;
- cenários de erro encontrados durante a validação devem ser transformados em testes de regressão para evitar retorno do problema;
- na camada de IA, o backend deve continuar controlando ferramentas, segurança, contexto e validação, evitando que respostas dependam apenas do modelo.

6. CONCLUSÃO

De forma geral, os testes executados indicam estabilidade nos principais fluxos do sistema após as correções realizadas. O processo mostrou a importância de planejar testes por camada, registrar evidências e manter critérios claros de sucesso para cada funcionalidade. A estratégia adotada permitiu validar o CashLens em diferentes pontos de risco: autenticação, integridade dos dados financeiros, importação, persistência, cálculos, API, frontend e IA. Com isso, o relatório final consolida não apenas a execução dos testes, mas também a contribuição do processo de testes para aumentar a confiabilidade do sistema.